Algorithm Engineering

Course Introduction and Getting Started With OpenMP

Mark Blacher

Friedrich Schiller University Jena

Winter Semester of 2026/27

Organization

- **One lecture per week** (asynchronously)
- **Slides**, **code** and **videos** are available here:
  https://cloud.uni-jena.de/s/yFiyK7BkdX8Fnin
- **Wednesdays** from **10:15 to 11:45** in **PC-Pool E019** (ground floor; Inselplatz 5; from October 28, 2026 on) you can **work on** the **assignments**, **projects**, **ask** me **questions** (**optional**)
- If you have **questions**, you can also **write** me an **e-mail**
  mark.blacher@uni-jena.de
- If something comes up that is relevant to everyone, I will write to everyone

Grading

No oral exam. **No written exam**.

The **following things determine** your **final grade**:

1. **Written answers** to the **Exam Assignments** and your **solutions** to **some small coding exercises**
2. **C++ Project** (If you know someone you want to do the project together, please do so, otherwise do it alone)
3. **Four page paper** about your **project** (I provide the double column LaTeX template)

Grade = *part 1* (40%) + *part 2* (40%) + *part 3* (20%)

You have time to fulfill everything until March 1, 2027.
Don't worry about the details of the individual parts for the time being, you will receive detailed information about them in the course of the lectures.

Create Your Own Repository

- You use **one repository for the entire course**
- Please **make three folders** in your repository (`answers`, `project`, `paper`)
- The **repository** is your **flagship**, make it **clean** and **clear**
- **By March 1, 2027**, email me the **content of your repository** (if it is not too big)
- You can also **add me to** the **repository** with the following email: `mark.blacher@uni-jena.de` (**give me** the **appropriate access rights**)
- Please **choose one** of the following repository providers:
  - **https://git.uni-jena.de/**
  - **https://gitlab.com/**
  - **https://github.com/**

Course Mantra

I hear and I forget.
I see and I remember.
I do and I understand.

Confucius

Motivation

Algorithm theory lacks the connection to real hardware. In addition to algorithmic complexity with its hidden constant factors, **hardware-specific implementation details determine the execution time of algorithms**. To close this gap between theory and practice, the field of algorithm engineering emerged. In terms of algorithm engineering, the challenge of making software faster is to **develop and implement algorithms that use the hardware efficiently**. Advances in computer architecture such as **multi-core processors, CPU caches, and SIMD instructions** opened up new opportunities for improving the performance of algorithms. In this course we learn how to exploit the hardware efficiently with C++ to speed up algorithms.

Course Overview

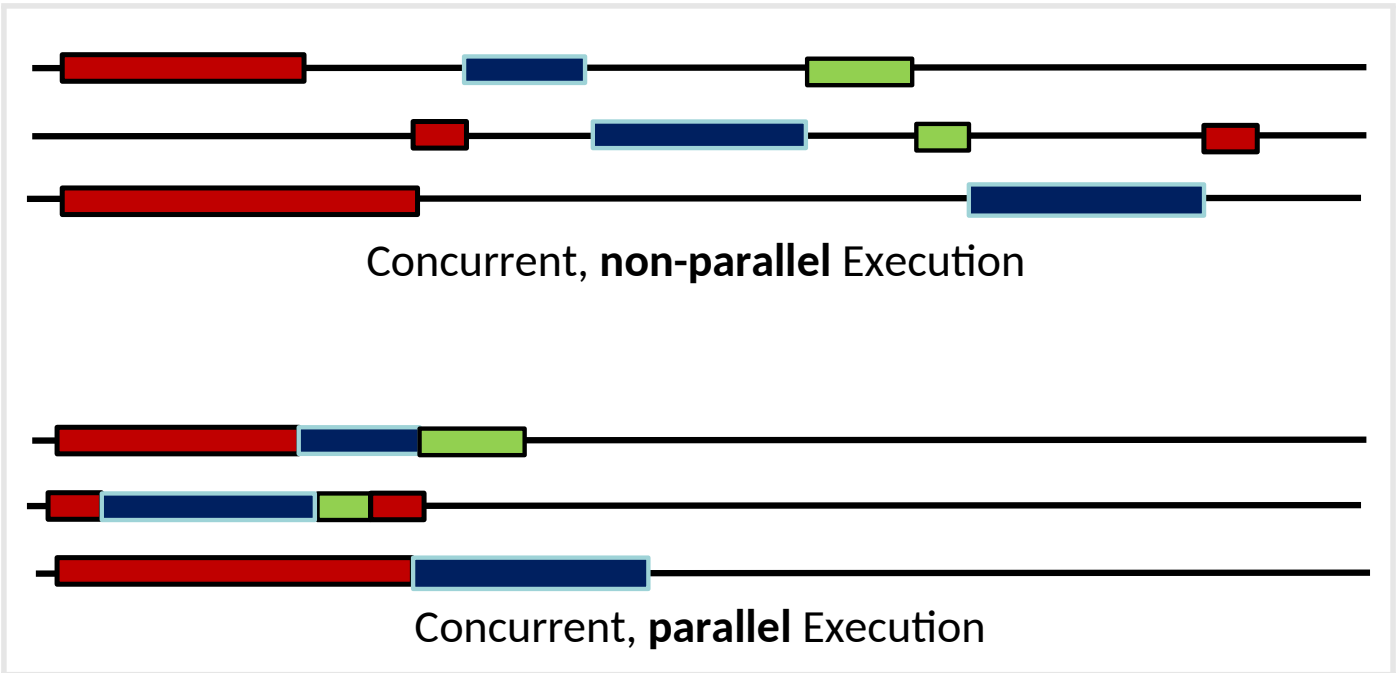
- Parallelization with **OpenMP**
- Compiling programs with the help of **CMake**
- Exploiting vector registers (**vectorization**)
- Writing **cache-friendly code**
- **Debugging** and **profiling**
- **Integrating C++ code in Python**
- Writing **efficient code for SSDs**

Recommended Literature

- https://chat.openai.com/auth/login
- http://chryswoods.com/beginning_c++/
- http://chryswoods.com/beginning_git/
- https://www.youtube.com/playlist?list=PLLX-Q6B8xqZ8n8bwjGdzBJ25X2utwnoEG
- https://www.agner.org/optimize/optimizing_cpp.pdf
- https://github.com/Kobzol/hardware-effects
- Computer Systems: A Programmer's Perspective (3rd Edition)
- C++ High Performance: Master the art of optimizing the functioning of your C++ code (2nd Edition)

Concurrency vs. **Parallelism**

A system is **concurrent** if it can support two or more actions *in progress* at the same time. A system is **parallel** if it can support two or more actions executing simultaneously.



Parallelism is a subset of concurrency

Why Parallel Computing?

The Free Lunch Is Over.

- The **number of transistors** on a microchip **doubles about every two years**. (Moore's Law)
- $power = performance^{1.74}$
- To keep up with the "doubling", we need multiple cores per CPU chip

Parallel Programming with **OpenMP**

OpenMP is . . .

- a set of **compiler directives**, **library routines** and **environment variables**
- easy to learn
- powerful
- **included in most C/C++ compilers** (and Fortran)

Hello World with OpenMP

hello.cpp

```cpp
#include <iostream>
#include <omp.h> // required for library routines

using namespace std;

int main() {
#pragma omp parallel // compiler directive
  {
    // omp_get_thread_num() is an OpenMP library routine
    auto thread_id = omp_get_thread_num();
    cout << "Hello from thread " << thread_id << endl;
  }
}
```

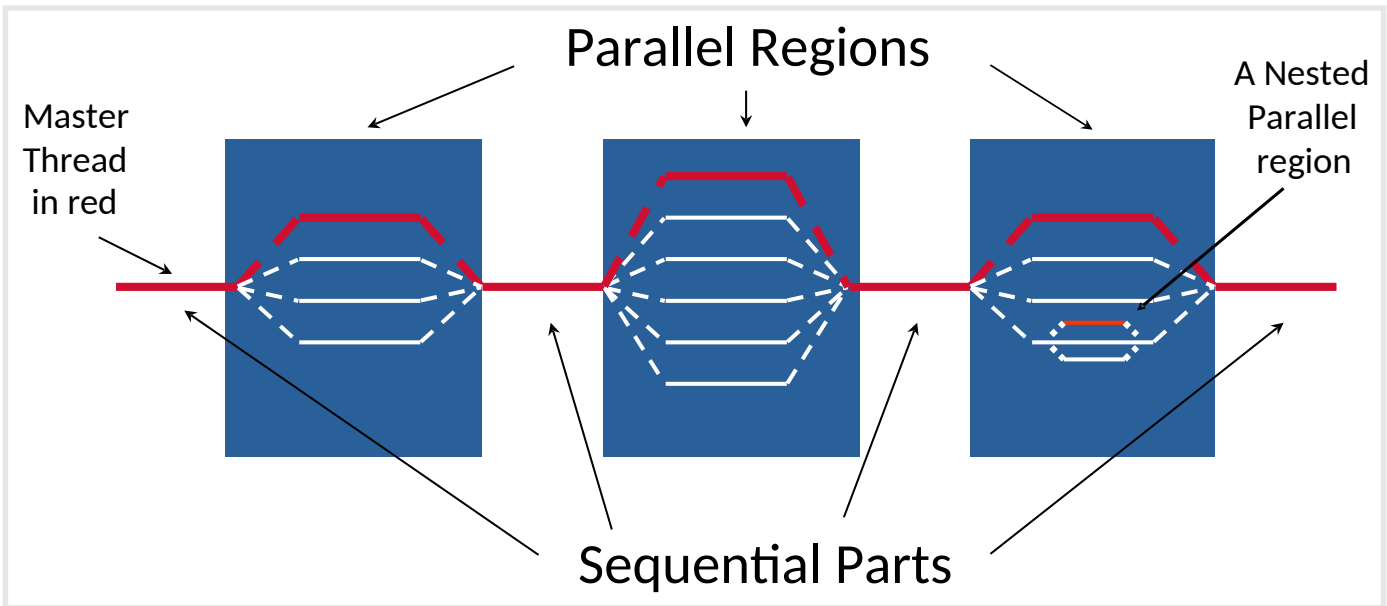
Terminal

```
$ g++ -fopenmp hello.cpp -o hello
$ OMP_NUM_THREADS=4 ./hello # environment variable
```

OpenMP Programming Model

Fork-Join Parallelism

Master thread spawns a **team of threads**. At a **subsequent point threads are joined** and the program resumes executing sequentially.



What the OpenMP Compiler Does

OpenMP code

```
1  #include <iostream>
2
3  int main() {
4  #pragma omp parallel num_threads(4)
5    { std::cout << "Hello World!" << std::endl; }
6  }
```

OpenMP compiler generates logically similar code to this

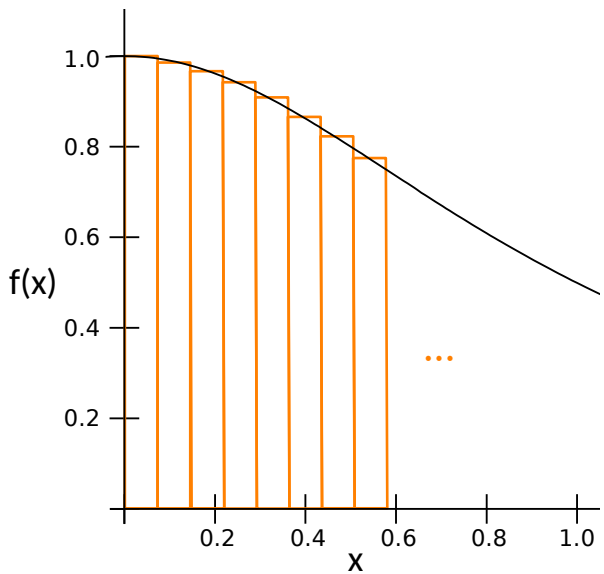
```
1  #include <iostream>
2  #include <thread>
3
4  void thunk() { std::cout << "Hello World!" << std::endl; }
5
6  int main() {
7    std::thread threads[4];
8    for (int i = 1; i < 4; ++i)
9      threads[i] = std::thread(thunk); // fork threads
10   thunk(); // master thread
11   for (int i = 1; i < 4; ++i)
12     threads[i].join(); // join threads
13 }
```

Estimating π

Mathematically, we know that:

$$\int_0^1 \frac{1}{1+x^2}\,dx = \frac{\pi}{4}$$

We can **approximate the integral** as a **sum of rectangles**.



Estimating π with Numerical Integration

pi_numerical_integration.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <omp.h>

using namespace std;

int main() {
  int num_steps = 100000000; // number of rectangles
  double width = 1.0 / double(num_steps); // width of a rectangle
  double sum = 0.0; // for summing up all heights of rectangles

  double start_time = omp_get_wtime(); // wall clock time in seconds
  for (int i = 0; i < num_steps; i++) {
    double x = (i + 0.5) * width; // midpoint
    sum = sum + (1.0 / (1.0 + x * x)); // add new height of a rectangle
  }
  double pi = sum * 4 * width; // compute pi
  double run_time = omp_get_wtime() - start_time;

  cout << "pi with " << num_steps << " steps is " << setprecision(17)
       << pi << " in " << setprecision(6) << run_time << " seconds\n";
}
```

PC-Pool E019





- Lenovo ThinkCentre M70q Gen 5 (Intel Core i7-14700T)
- 8 P-cores (2 threads/P-core) + 12 E-cores
- 32 GB RAM
- Intel UHD Graphics 770

You can **compile your programs with** g++

Terminal

```
$ g++ -fopenmp hello.cpp -o hello
$ ./hello
```

Compiler, IDE Recommendations

You **need** a **compiler** that **supports OpenMP**

- **Linux:** GNU C++ Compiler, Install on Ubuntu
- **Windows:** MinGW 19.0 (or 20.0) Distro from https://nuwen.net/mingw.html
- **Mac:** GNU compiler collection, Install Homebrew , Install gcc

You can also get for free **Intel oneAPI Toolkit**,
Click here for installation and more information

IDE recommendation: Clion

Coding Warmup

1. **Compile and run** the programs of **slides 12 and 14**
2. **Create a parallel version** of the π program (**slide 16**)

In addition to `#pragma omp parallel`, you will need the runtime library routines:

- `omp_get_num_threads()` *// number of threads*
- `omp_get_thread_num()` *// thread id*
- `omp_get_wtime()` *// wall-clock time in seconds*

3. **Watch** the **times** your implementation needs with **different numbers of threads**
4. Try compiling with **optimization flag** `-Ofast`

Possible Solution for the Warmup π Program

pi_openmp_v1.cpp

```cpp
#include <iomanip>
#include <iostream>
#include <omp.h>

using namespace std;

int main() {
  int num_steps = 100000000; // number of rectangles
  double width = 1.0 / double(num_steps); // width of a rectangle
  double sum = 0.0; // for summing up all heights of rectangles
  double start_time = omp_get_wtime(); // wall clock time in seconds
  omp_set_num_threads(4); // setting the desired number of threads in the parallel region
#pragma omp parallel // parallel region starts here
  { // idea for parallelisation: split up loop iterations between threads
    int num_threads = omp_get_num_threads(); // used as increment in for loop
    int thread_id = omp_get_thread_num();
    double sum_local = 0.0; // for summing up heights locally
    for (int i = thread_id; i < num_steps; i += num_threads) {
      double x = (i + 0.5) * width; // midpoint
      sum_local = sum_local + (1.0 / (1.0 + x * x)); // add new height
    }
#pragma omp atomic // only one thread performs the update of the sum
    sum += sum_local;
  } // parallel region ends here
  double pi = sum * 4 * width; // compute pi
  double run_time = omp_get_wtime() - start_time;
  cout << "pi with " << num_steps << " steps is " << setprecision(17) << pi
       << " in " << setprecision(6) << run_time << " seconds\n";
}
```

Estimating π using Monte Carlo

pi_monte_carlo.cpp

```cpp
#include <iostream>
#include <omp.h>
#include <random>

using namespace std;

int main() {
  unsigned int seed = 0;
  default_random_engine re{seed};
  uniform_real_distribution<double> zero_to_one{0.0, 1.0};

  int n = 100000000; // number of points to generate
  int counter = 0; // counter for points lying in the first quadrant of a unit circle
  auto start_time = omp_get_wtime(); // omp_get_wtime() is an OpenMP library routine

  // compute n points and test if they lie within the first quadrant of a unit circle
  for (int i = 0; i < n; ++i) {
    auto x = zero_to_one(re); // generate random number between 0.0 and 1.0
    auto y = zero_to_one(re); // generate random number between 0.0 and 1.0
    if (x * x + y * y <= 1.0) { // if the point lies in the first quadrant of a unit circle
      ++counter;
    }
  }

  auto run_time = omp_get_wtime() - start_time;
  auto pi = 4 * (double(counter) / n);

  cout << "pi: " << pi << endl;
  cout << "run_time: " << run_time << " s" << endl;
  cout << "n: " << n << endl; }
```

Exam Assignments

- **Select** one **slide** from the lecture, research more about the topic, and **report on it**.
- **Read Chapter 1** from Computer Systems: A Programmer's Perspective.
  http://csapp.cs.cmu.edu/2e/ch1-preview.pdf
  - **Discuss two things** you find particularly interesting.
- Do the **coding warmup** on **slide 19**. (optional)
- **Parallelize the program** of **slide 21** with OpenMP.